Failure (a bad split, and a k comparison) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `chunking_practice.py` (Failure section): it's tempting to treat "just raise `k`" as a fix once it appears to work on a tiny test document — this exercise makes you watch that assumption fail at real scale before you carry it into the Build Task. **If not:** the Build Task's knowledge base would inherit a `k`-bump habit that quietly stops working the moment the corpus grows past a few dozen chunks.

All examples below use:

```
# chunking_practice.py — Failure section
TEST_DOCUMENT = (
    "Our team ships a small internal newsletter every month. "
    "The Treaty of Lisbon was signed on 13 December 2007 in Lisbon, Portugal, "
    "marking a major change to how the European Union is governed. "
    "Most readers skip the history section and go straight to the recipe "
    "at the bottom."
)
QUESTION = "When and where was the Treaty of Lisbon signed?"
```

Basic Version

Approach 1 — the direct way

```
# chunking_practice.py — Failure section
import math
import time
from openai import OpenAI

client = OpenAI()

def chunk_by_chars(text, chunk_size):
    chunks = []
    start = 0
    while start < len(text):
```

```
        chunks.append(text[start:start + chunk_size])
        start += chunk_size
    return chunks

def get_embedding(text):
    response = client.embeddings.create(
        model="text-embedding-3-small", input=text
    )
    return response.data[0].embedding

def cosine_similarity(a, b):
    dot = 0.0
    for x, y in zip(a, b):
        dot = dot + x * y
    squares = 0.0
    for x in a:
        squares = squares + x * x
    norm_a = math.sqrt(squares)
    squares = 0.0
    for x in b:
        squares = squares + x * x
    norm_b = math.sqrt(squares)
    return dot / (norm_a * norm_b)

def retrieve(question, chunks_with_embeddings, k):
    question_embedding = get_embedding(question)
    scored = []
    for chunk_text, embedding in chunks_with_embeddings:
        score = cosine_similarity(question_embedding, embedding)
        scored.append((score, chunk_text))
    scored.sort(reverse=True)
    return scored[:k]

chunks = chunk_by_chars(TEST_DOCUMENT, 40)
print("chunks:")
for i, chunk in enumerate(chunks):
    print(i, ":", repr(chunk))

chunks_with_embeddings = []
for chunk in chunks:
    chunks_with_embeddings.append((chunk, get_embedding(chunk)))

print()
print("k=1 result:")
for score, chunk_text in retrieve(QUESTION, chunks_with_embeddings, k=1):
    print(score, repr(chunk_text))
```

```
print()
print("timing at k=1, k=3, k=10:")
for k in (1, 3, 10):
    start = time.perf_counter()
    results = retrieve(QUESTION, chunks_with_embeddings, k)
    elapsed = time.perf_counter() - start
    print("k =", k, "| time:", round(elapsed, 4), "seconds")
    for score, chunk_text in results:
        print("   ", round(score, 3), repr(chunk_text))
```

Expected output (shape): the fact's sentence gets sliced across 40-character chunks, mid-word: one chunk is `' signed on 13 December 2007 in Lisbon, P'` and the next starts `'ortugal, marking...'`. At `k=1`, the best you can get is the date and half the place — "Portugal" is cut in two. As `k` grows to 3 and then 10, more surrounding chunks come back and the full fact becomes reconstructable — but the timing barely changes at this scale, because comparing a question against a handful of chunks one by one is already close to instant.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — `build_index()` / `retrieve()`, factored apart

```
# chunking_practice.py — Failure section
import math
import time
from openai import OpenAI

client = OpenAI()


def chunk_by_chars(text: str, chunk_size: int) -> list[str]:
    chunks: list[str] = []
    start = 0
    while start < len(text):
        chunks.append(text[start:start + chunk_size])
        start += chunk_size
    return chunks


def get_embedding(text: str) -> list[float]:
    response = client.embeddings.create(
        model="text-embedding-3-small", input=text
    )
    return response.data[0].embedding


def cosine_similarity(a: list[float], b: list[float]) -> float:
    dot = 0.0
    for x, y in zip(a, b):
        dot = dot + x * y
    squares = 0.0
    for x in a:
        squares = squares + x * x
    norm_a = math.sqrt(squares)
    squares = 0.0
    for x in b:
        squares = squares + x * x
    norm_b = math.sqrt(squares)
    return dot / (norm_a * norm_b)
```

```
def retrieve(
    question: str, chunks_with_embeddings: list[tuple[str, list[float]]], k: int
) -> list[tuple[float, str]]:
    question_embedding = get_embedding(question)
    scored: list[tuple[float, str]] = []
    for chunk_text, embedding in chunks_with_embeddings:
        score = cosine_similarity(question_embedding, embedding)
        scored.append((score, chunk_text))
    scored.sort(reverse=True)
    return scored[:k]


def build_index(text: str, chunk_size: int) -> list[tuple[str, list[float]]]:
    chunks = chunk_by_chars(text, chunk_size)
    results = []
    for chunk in chunks:
        results.append((chunk, get_embedding(chunk)))
    return results


def main() -> None:
    chunks_with_embeddings = build_index(TEST_DOCUMENT, 40)

    print("k=1 result:")
    for score, chunk_text in retrieve(QUESTION, chunks_with_embeddings, k=1):
        print(f"  {score:.3f}  {chunk_text!r}")

    print("\ntiming at k=1, k=3, k=10:")
    for k in (1, 3, 10):
        start = time.perf_counter()
        results = retrieve(QUESTION, chunks_with_embeddings, k)
        elapsed = time.perf_counter() - start
        print(f"k = {k} | time: {elapsed:.4f} seconds")
        for score, chunk_text in results:
            print(f"    {score:.3f}  {chunk_text!r}")


if __name__ == "__main__":
    main()
```

Difference from Basic: full type hints, and `build_index()` separates "turn raw text into a searchable index" from `retrieve()`'s "search an existing index" — a real pipeline builds the index once and searches it many times, so keeping those responsibilities apart matters even in a small script.

Approach 2 — timing at real scale, not toy scale

Story: the timing above barely moves between `k=1`, `k=3`, and `k=10` — and that result is easy to over-generalize from. It only barely moves because `retrieve()` compares the question against a *handful* of chunks; the cost that actually scales with corpus size was always there, just invisible at this size. **If not:** "raising `k` is basically free" would look true right up until the knowledge base actually grew.

The large corpus has 1,365 chunks. Approach 1's `build_index()` would make 1,365 separate embedding calls — minutes of waiting. So this approach embeds them in batches, the Basic exercise's `get_embeddings_batch` idea: one call takes a list, and returns one vector per text, in the same order. One call accepts up to 2,048 texts, so the list is sent in slices of 1,000.

```
# chunking_practice.py — Failure section
BATCH_SIZE = 1000   # one embeddings call accepts up to 2,048 texts


def get_embeddings_batch(texts: list[str]) -> list[list[float]]:
    vectors: list[list[float]] = []
    for start in range(0, len(texts), BATCH_SIZE):
        response = client.embeddings.create(
            model="text-embedding-3-small",
            input=texts[start:start + BATCH_SIZE],
        )
        for item in response.data:
            vectors.append(item.embedding)
    return vectors


def build_index_batched(
    text: str, chunk_size: int,
) -> list[tuple[str, list[float]]]:
    chunks = chunk_by_chars(text, chunk_size)
    vectors = get_embeddings_batch(chunks)
    results = []
    for chunk, vector in zip(chunks, vectors):
        results.append((chunk, vector))
    return results


def build_large_corpus(document: str, repeats: int) -> str:
    return document * repeats


large_document = build_large_corpus(TEST_DOCUMENT, repeats=200)
large_index = build_index_batched(large_document, chunk_size=40)
```

```
print(f"large corpus: {len(large_index)} chunks")

print("timing at k=1, k=3, k=10 (large corpus):")
for k in (1, 3, 10):
    start = time.perf_counter()
    retrieve(QUESTION, large_index, k)
    elapsed = time.perf_counter() - start
    print(f"k = {k} | time: {elapsed:.4f} seconds")
```

Expected output (shape): `large corpus: 1365 chunks`, built with 2 embedding calls instead of 1,365. Where the small corpus showed almost no timing difference across `k` values, the large corpus's `retrieve()` calls all take noticeably longer than any of the small-corpus calls did — because `retrieve()` scores *every* stored chunk against the question no matter what `k` is; `k` only controls how many of those already-computed scores get returned. (Each `retrieve()` call also makes one embedding call for the question, so the time includes that network wait.) The cost that actually scales with corpus size was always there — the earlier, tiny example was just too small to show it. This is exactly why a real vector store (Chroma, FAISS, a hosted service) doesn't do this brute-force comparison at all once a knowledge base grows past a few thousand chunks — it builds an index ahead of time so a query only touches a small, likely-relevant fraction of the stored vectors, not every single one.

Approach 3 — the real fix for the split fact: overlap, not a bigger k

Story: fixing the "half the fact went missing" bug is not the same problem as picking a good `k` — `k` decides how many results come back, but where the boundary falls is decided by the chunker. **If not:** raising `k` would keep looking like the fix, right up until a bigger corpus made it stop working (Approach 2's lesson) — overlap removes the cause instead of hiding the symptom.

```
# chunking_practice.py — Failure section
def chunk_by_chars_with_overlap(
    text: str, chunk_size: int, overlap: int,
) -> list[str]:
    chunks: list[str] = []
    step = chunk_size - overlap  # why: less than chunk_size -- chunks overlap
    start = 0
    while start < len(text):
        chunks.append(text[start:start + chunk_size])
        start += step
    return chunks


def build_overlap_index(
    text: str, chunk_size: int, overlap: int,
) -> list[tuple[str, list[float]]]:
    chunks = chunk_by_chars_with_overlap(text, chunk_size, overlap)
```

```
    results = []
    for chunk in chunks:
        results.append((chunk, get_embedding(chunk)))
    return results


overlap_index = build_overlap_index(TEST_DOCUMENT, chunk_size=120, overlap=80)

print("k=1 result, WITH overlap:")
for score, chunk_text in retrieve(QUESTION, overlap_index, k=1):
    print(f"  {score:.3f}  {chunk_text!r}")
```

Expected output (wrapped here to fit the page; your score will differ):

```
k=1 result, WITH overlap:
  0.563  'er every month. The Treaty of Lisbon was signed on 13 December
  2007 in Lisbon, Portugal, marking a major change to how t'
```

The fact sentence, `"The Treaty of Lisbon was signed on 13 December 2007 in Lisbon, Portugal"`, is 71 characters long. Any text up to `overlap` characters long always fits whole inside some chunk, so `overlap=80` (with `chunk_size=120`) guarantees one chunk holds the whole sentence — here, chunk 1 of 7. A smaller overlap is a gamble: `chunk_size=40, overlap=15` gives 11 chunks and **none** holds even the date and place together. With the right overlap, `retrieve(..., k=1)` alone returns the complete fact — no need for `k=3` or `k=10` to reconstruct it from separate pieces.

Difference from Approach 1, and between Approaches 2/3: Approach 1 measures timing at a scale too small to reveal that brute-force cost actually grows with corpus size — Approach 2 makes that cost visible, and explains why real vector stores don't do this same brute-force comparison at scale. Approach 3 tackles a different question entirely: not "how do we search faster," but "how do we stop losing half the fact in the first place" — and shows that the answer isn't `k`, it's the chunker.

Which one should you actually use? Both matter, but they answer different questions. For "the search is too slow," reach for a real vector index (Approach 2's lesson) once your chunk count is large enough for brute force to actually hurt — not before, since brute force over a small corpus is simpler and genuinely fine. For "the answer is missing half a fact," reach for overlap (Approach 3) — not a bigger `k` — since `k` only ever improves the odds of the workaround, while overlap removes the actual cause. Mixing these two up (treating a `k` bump as if it fixes the boundary-split problem) is the exact trap this exercise exists to catch you doing once, on purpose, before it happens to you by accident in a real project.